

What's hard about Rust is not exclusively the syntax, but systems programming in general, which is what this language was designed to excel at. There is a considerable amount of computer and systems theory behind it. If we want to learn it properly, we must at least have a generalized understanding of concepts such as memory allocation, pointers, borrowing, and data structures.

That said, for those with a background in systems programming, learning Rust will hopefully be a smooth transition, as many of the low-level concepts are similar, if not the same, and some syntactic elements are alike.

In this segment, we will provide an introductory overview of the Rust language. We will also provide enough theory to understand what we're programming, although we will not dive too deep into the Rustacean waters since this is meant to be an introduction to the language.

We'll be using VS Code as well as PowerShell throughout this article.

§

Installation

For this segment, we will need to install three main components:

- The Rust programming language.
- Visual Studio Code.
- The Visual Studio Code Rust extension.

We will also install some packages, which will come later when we get to the package manager.

1. Rust

There are two main methods we can use to install the Rust programming language:

- Via the `rustup` installer.
- Via standalone installers in the form of tarballs (`.tar.gz`) for Unix-like environments, a Windows installer (`.msi`), or a macOS installer (`.pkg`).

We'll stick `rustup`, the recommended method in the official documentation; it has a 6-week rapid release process and supports various platforms. The complete installer documentation can be found [here](#).

To install Rust, we will follow the steps below:

1. Head to the [Rust language installation page](#).
2. Download the 64-bit executable.
3. Run the executable.
4. Take note of the Cargo home directory displayed in the installation prompt (*usually located in* `C:\Users\username\.cargo`).
5. Wait for the following prompt: `You can uninstall at any time...`
6. Hit `Enter`.
7. Wait for the following prompt: `Rust is installed now. Great!`
8. Hit `Enter`.
9. Open a new shell and input the following: `rustc --version`
10. If the installed `rustc` version appeared, the installation succeeded.

If, instead, we got an error such as `rustc command not found`, it most probably means that the `.cargo/bin` directory was not added to the `Path` environment variable. To fix this, we will need to add it manually by following the steps below:

1. Open the Windows run command window by typing `WIN+R`.

2. Input the following: `"C:\Windows\system32\rundll32.exe" sysdm.cpl,EditEnvironmentVariables`
4. Select `Path`.
5. Click `Edit...`.
6. Click `New`.
7. Input the following: `C:\Users\username\.cargo\bin`.
8. Click `OK`.
9. Click `OK`.
10. Try to execute the `rustc --version` command again.

2. Microsoft C++ Build Tools

To compile Rust code, we will need to install the Microsoft C++ Build Tools. It is probable that we already have this installed. If we don't, we can follow the steps below:

1. Head to the [Microsoft Visual Studio Downloads Page](#).
2. On *All Downloads*, look for `build tools`.
3. Download the *Build Tools for Visual Studio 2022* installer.
4. Install the Microsoft C++ Build Tools.

3. VS Code

If we don't yet have VS Code installed, we can get it from the [official downloads page](#). We need to select the Windows 8, 10, 11 executable and wait for it to download. When the installation is complete, we can verify by opening the Visual Studio Code application directly from the Windows start menu. A detailed configuration guide for VS Code is out of the scope of this article but can be consulted on the [VS Code official documentation site](#).

4. Rust VS Code extension

Once we have Rust and VS Code installed, we will proceed to install the Rust VS Code Extension:

1. Open VS code and head to the Extensions menu in the left panel. We can also open the Extensions menu by using the shortcut `Ctrl + Shift + X` or by opening the command palette by typing `F1` and searching for *Extensions: Install Extensions*.
2. We will search for `rust-analyzer`, maintained by *rust-lang.org*, and install and enable it. We can also get the extension using [this link](#).

Now that everything's in place, we're ready to start configuring our working environment.

§

Creating a project

The first thing to do after installation is to create a Rust project. This is handled by the `cargo` package manager.

There are two options for creating a new project:

- Creating a new project folder.
- Using an existing folder as the project folder.

If we want to create a project from scratch, we can first head to our project directory:

CODE

```
cd Projects
```

We will then initialize our project:

CODE

```
cargo new rust-for-beginners
```

Conversely, if we want to use the current directory as the project folder, we can simply change directories to our target folder and initialize it as our project:

CODE

```
cd rust-for-beginners  
cargo init
```

This command will create two components:

- A `src` folder: Contains our project's source code.
- A `Cargo.toml` file: This is the project's manifest and contains metadata needed to compile our package, such as the project name and required dependencies.

Once we're at it, we can mention that the `.toml` file format is relatively popular among the Rust developer ecosystem, notably used by Cargo, mainly for configuration purposes.

We can optionally install a VS Code extension that will make our life easier when dealing with this file format:

1. Open VS code and head to the Extensions menu in the left panel. We can also open the Extensions menu by using the shortcut `Ctrl + Shift + X` or by opening the command palette by typing `F1` and searching for *Extensions: Install Extensions*.
2. To install and enable it, we will search for `Even Better TOML`, maintained by *tamasfe*. We can also get the extension by using [this link](#).

This extension is backed by [taplo](#) and will support TOML syntax.

1. The project manifesto

If we open our `Cargo.toml` file, we will see that it contains some boilerplate metadata about our newly-created project:

- `package` : Defines a package.
 - `name` : The name of the package.
 - `version` : The version of the package (*not the Rust release*).
 - `edition` : The Rust edition.
- `dependencies` : Package library dependencies.

A `.toml` file general structure consists of sections and key-value pairs, similar to a Python dictionary; a `Cargo.toml` file section groups similar variables together, depending on their functionality.

The complete set of sections and key-value pairs can be consulted [here](#).

Once we start including dependencies in our project, we will also start working on our manifesto. Once the file is generated, the `Cargo.toml` manifesto is meant to be edited by us.

2. Including dependencies

As we mentioned, Rust dependencies are handled by Cargo. There are three main concepts that we should be aware of:

- **Module / Struct:** A module is a way to organize code and data into logical units. A module can contain functions, structs, enums, constants, types, and other modules.
- **Crate:** A crate is a compilation unit in Rust that produces a binary or a library. It contains one or more modules that define functions, structs, enums, traits, and other Rust language constructs. A crate can be compiled into a binary executable or a library that other programs can use.
- **Library:** A library is a special kind of crate intended to be used by other programs. It provides a collection of reusable code that can be linked to other Rust programs. A library can be either a dynamic library (`.so` or `.dll`) or a static library (`.a` or `.lib`). Rust libraries can be organized into modules similar to namespaces in other languages.
- **Package:** A package is a set of one or more crates built and tested together. A package contains a `Cargo.toml` file that specifies the package's name, version, authors, dependencies, build script, and other metadata. A package can be published to the Rust package registry (crates.io) to be used by other Rust developers.

Let us head to VS Code and open our `main.rs` source file. Once we open it, we'll notice that two new items have been created in our project folder:

- A `Cargo.lock` file: Contains exact information about our dependencies. It is maintained by Cargo and should not be manually edited.
- A `target` folder: Where the output of a build is stored.
 - A `debug` folder: Where the debug build is stored.
 - A `.rustc_info.json` file: Contains metadata about the compilation process and the target platform for which the code was compiled.
 - A `CACHEDIR.TAG` file: Marks a directory as a cache directory. Tools that recognize the signature inside this file will treat the directory as a cache directory and may take appropriate action, such as excluding it from backups or indexing.

We'll discuss these files & folders in more detail as we move on. For now, we will focus on the `main.rs` file.

We will start by importing the following modules into our main file:

- `std::io` : Standard input/output module.
- `std::cmp::Ordering` : Comparison module.
- `rand::Rng` : Random number generator module.

CODE

```
use std::io;
use std::cmp::Ordering;
use rand::Rng;
```

If we pay close attention, we can see that we used the keyword `use` to import a library followed by a double colon `::`, the required module(s), nested imports in the case of `std::io` enclosed in curly braces `{}`, and a semicolon `;` termination.

We can also see that two alerts pop up:

- An unused import alert for `std::io` : Rust knows that we have not used the module in our code; hence it raises an alert.
- An unresolved import for `rand` : We have not specified a version for the `rand` library; hence Rust raises an error.

The first alert does not require further action; we will eventually use this module. For the second one, we will head to our `Cargo.toml` manifesto and include the library version under `[dependencies]` :

CODE

```
[dependencies]
rand = "0.8.5"
```

Once we do this, the error should disappear.

3. The main function

As we will see later, every executable Rust program requires a `main` function. The main function serves as the entry point to our program.

Whenever we create a new Rust project, this main function will be written by default in our `main.rs` file:

CODE

```
fn main() {
    println!("Hello, world!");
}
```

If we leave this as is, our program will simply print `Hello, world!` to `stdout` .

4. Compiling the project

Compiling and running a Rust project can be done using Cargo and consists of 3 main steps:

- Navigate to the project directory.
- Build the project.
- Run the project.

CODE

```
# Navigate to the project directory
cd rust-for-beginners

# Build the project
cargo build

# Run the project
cargo run
```

If the project was built successfully, we'll get the build confirmation as well as the string from our `println!()` statement inside our `main` function:

OUTPUT

```
Compiling rust-for-beginners v0.1.0 (C:\Users\username\Documents\rust-for-beginners)
  Finished dev [unoptimized + debuginfo] target(s) in 0.29s

Hello, world!
```

Else, we'll get a build error:

OUTPUT

```
Compiling rust-for-beginners v0.1.0
error: Compilation Error Message
```

§

Commenting

Comments are fragments of code that are not included for execution during compilation and running. There are multiple ways to comment in Rust, although we will only review the two most used:

- **Line Comment:** Single line.
- **Block Comment:** Multiline.

1. Line comment

We can comment on a single line by prepending our comment with a double slash `//`:

CODE

```
// This is a single line comment.
// Which can also be continued in the next line.
```

2. Block comment

We can comment on a block by enclosing our comment in slash asterisk symbols `/* */`:

CODE

```
/*
This is a multiline comment
where we can include multiple lines
without having to comment each line.
*/
```

Pointers, references, and ownership

Before diving deeper, it's worth spending some time understanding what pointers are and how Rust's memory, ownership, and references work. This is because these concepts are key in understanding not just Rust's architecture but any low-level language such as C and C++; plus, it will save us a lot of frustration when we get to variables and data types.

1. Pointers

Formally, a pointer is a derived data type that can store the memory address of other variables. It allows us to manipulate and reference data stored in memory by directly accessing its location in memory.

Let us represent a generic block of memory using an array-like structure:

Address	Value
0x0000	01100110
0x0001	00100001
0x0002	10110100
0x0003	11001010

TABLE 1: REPRESENTATION OF A MEMORY BLOCK USING AN ARRAY-LIKE STRUCTURE

Where:

- The **address** is the location inside the memory block.
- The **value** is the data stored at that particular address.

In this example, the memory block contains four consecutive locations starting at address 0x0000. Each memory location stores a byte of data represented by a sequence of 8 bits. This example's values are binary, but they can also be represented in hexadecimal or decimal notation.

If we convert our binary sequences to decimal numbers, we get the following:

Address	Value
0x0000	102
0x0001	33
0x0002	180
0x0003	202

TABLE 2: BINARY CHARACTERS CONVERTED TO DECIMAL NUMBERS

If we look closer, the addresses in our memory block are simply numbers represented in hexadecimal form; the `0x` character preceding all the address values denotes we're talking about a hexadecimal value, where the number following would be the actual hexadecimal value.

To convert it to decimal, we simply solve for the following:

$$0x0001 = 0x16^3 + 0x16^2 + 0x16^1 + 1x16^0$$

Simplifying:

$$0x0001 = 0x4096 + 0x256 + 0x16 + 1x1$$

Thus, our decimal number would be 1.

Following this logic, we could define a value in our memory block as an address; this is the definition of a pointer: a value that happens to be an address in memory.

If we substitute the value corresponding to the memory address 0x0002 for another address within the same block and convert the remaining binary representations to hexadecimal 0x notation, we would end up with the following:

Address	Value
0x0000	0x0066
0x0001	0x0021
0x0002	0x0001
0x0003	0x00CA

TABLE 3: MEMORY BLOCK CONTAINING A POINTER AT ADDRESS 0x0002, REFERENCING ADDRESS 0x0001, OR VALUE 0x0021

Now we have a pointer, but why is this relevant? Well, in Rust, we have three types of pointers:

- References
- Raw pointers
- Smart pointers

We'll focus on the most common type: references.

2. References

In Rust, a reference is a type of pointer that lets us borrow values without taking ownership. This means that we can include an additional layer of abstraction (e.g., *referencing a value without directly interacting with it*), which allows us to borrow values without copying them. Borrowing rules are validated on compile time by the borrow checker.

This can be much more efficient than making copies of large data structures and is especially important in performance-critical code, where reducing memory usage can improve program speed and reduce resource consumption.

As we will see, references in Rust are created using the ampersand `&` operator. For example, if we want to define a reference to the `str` data type and assign it to a variable, we can use the following syntax:

CODE

```
fn main() {  
    let _mystring:&str = "string";  
}
```

Not to worry. We'll dive deeper into variable declarations when we get to variables.

3. Ownership

We've already seen an example of borrowing using a data type reference (*i.e.*, the `str` data type has an owner, and we borrow it from that owner to define a new variable). This concept is part of a broader idea called ownership.

Ownership in Rust is based on the concept of a unique owner for each value. When a value is created, it is associated with a unique owner responsible for managing the value's memory allocation and lifetime. The owner can be a variable, a function, or a data structure.

Apart from borrowing, we can also transfer ownership using Rust's `move` semantics; when a value is assigned to another owner, the original owner is said to have "*moved*" the value. The new owner takes over responsibility for the value's memory allocation and lifetime, and the original owner becomes invalid.

With this concept, we've also introduced another key idea in Rust: memory safety.

When an owner goes out of scope or is no longer needed, Rust automatically frees the memory associated with the value. This ensures that Rust programs are memory safe, efficient, and free from common low-level programming errors such as dangling pointers, use-after-free, and data races. In short, ownership ensures that there is always exactly one owner for the value and that the owner is responsible for managing the value's memory allocation and lifetime.

Now that we have a general understanding of pointers, references, and ownership, we can discuss variables.

§

Variables

There are several different variable types depending on their structure. Generally, we use `let` to define a variable under the current scope. There are also static variables, although we won't be covering them here.

We must remember that Rust cares about unused variables; if we declare a variable and don't reference it anywhere in our code, the compiler will throw an error unless we prepend our variable name with an underscore `_`. This tells Rust that the defined variable is meant to be used as a placeholder, and thus the compiler will ignore it:

CODE

```
fn main() {  
    let _myplaceholder:i32 = 34;  
    let myplaceholder:i32 = 34;  
}
```

The first declaration won't throw an error if left unreferenced, while the second one will.

We can also use underscores `_` in between digits when dealing with numeric variables to denote thousand separators; this will increase legibility while not having any consequence during compilation:

CODE

```
fn main() {
    // Using underscores as separators
    let mynum_immutable = 7_000_000;
    println!("{}", mynum_immutable);
}
```

OUTPUT

```
7000000
```

As expected, the value will be printed without the underscore characters.

If we recall, we installed a Rust extension for VS Code. This extension provides helpful features such as data type inference upon declaring a variable. We cannot see it directly in our code, but the extension is using the compiler to include the following:

CODE

```
fn main() {
    // Using underscores as separators
    let mynum_immutable: i32 = 7_000_000;
    println!("{}", mynum_immutable);
}
```

Note the `i32` specification after the name of our variable.

We can also assign multiple variables in a single statement using tuples (*we'll discuss tuples later in the segment*):

CODE

```
fn main() {
    // Multiple variable assignment
    let (myvar1, myvar2, myvar3) = (1, 2, 3);
    println!("{}", myvar1, myvar2, myvar3);
}
```

OUTPUT

```
1 2 3
```

1. Immutable variables

Variables in Rust are immutable by default; when a variable is immutable, we can't change a value once a value is bound to a name. This is the default behavior for security purposes; we don't have to keep track of the variables and how they change throughout our program if they cannot be changed in the first place unless we explicitly decide to do so.

The simplest way to define a mutable variable inside a function is as follows:

CODE

```
fn main() {  
    let mystring_immutable:&str = "Hello World";  
}
```

Where:

- `mystring_immutable` is the variable name.
- `&str` is the borrowed form of the `str` type.
- `Hello World` is the value assigned to the variable.

2. Mutable variables

Mutable variables can change throughout our code. The simplest way to define a mutable variable inside a function is as follows:

CODE

```
fn main() {  
    let mut mystring_mut:String = String::new();  
}
```

Where:

- `mut` refers to the variable being mutable.
- `mystring_mut` refers to the variable name.
- `String` refers to the data type.
- `new()` tells Rust to create this as a new variable.

3. Scope & blocks

As we have mentioned, variable bindings in Rust have a scope. Scopes are important since they indicate to the compiler when borrows are valid, when resources can be freed, and when variables are created or destroyed.

A block or block expression is a control flow expression and anonymous namespace scope for items and variable declarations.

A block can be defined by using curly brackets:

CODE

```
fn main() {
    // Defining a block
    let var_outsideblock = 49;
    {
        // Inside a block
        let var_insideblock = 7;
        println!("Inside 1: {}", var_insideblock);
        println!("Inside 2: {}", var_outsideblock);
    }

    // Outside a block
    println!("Outside: {}", var_outsideblock);
}
```

Variables defined inside a block using `let` will not be accessible outside that block. However, variables defined outside a block using `let` will be accessible from within a given block.

This concept also applies to functions, where we can define a variable under the current scope that will not be accessible outside of that function.

§

Constants

Constants in Rust are different from immutable variables in that they are computed at compilation time (*and can be used in other compile-time computation*), and hence the run time is faster, as it does not need to compute it again; immutable variables are always computed at run time (*all `const` occurrences will be replaced by the value assigned on compilation*). Additionally, immutable variables are defined under the current scope (*are not defined globally*), while constants are global.

Let us elaborate further by using an example:

CODE

```
// Define constant outside main function
const HELLO: &str = "Constant Hello";

fn main() {
    // Print constant from inside main function
    println!("{}", HELLO);
}
```

As we might have noticed, we define a `const` using uppercase letters.

OUTPUT

```
Constant Hello
```

Conversely, if we try to do the same for an immutable variable, the compiler won't even let us declare it outside our function:

CODE

```
// Define an immutable variable outside main function
let mystring_immutable = 7;

fn main() {
    println!("{}", mystring_immutable);
}
```

OUTPUT

```
consider using `const` or `static` instead of `let` for global variables
```

§

Printing

There are multiple ways we can use to print values in Rust. Below are the two most commonly used:

- `print!()`
- `println!()`

We might have noticed the exclamation mark `!` at the end of each statement because both printing methods are macros (*we'll discuss macros later on*).

Printing a string literal in Rust can be achieved using the `print!()` macro without any additional arguments:

CODE

```
fn main() {
    // Print a string in same line
    print!("Hello ");
    print!("World");
}
```

OUTPUT

```
Hello World
```

This method will not include a newline at the end of each print statement, so it's sometimes best to use the `println!()` macro instead:

CODE

```
fn main() {  
    // Print a string in newline  
    println!("Hello");  
    println!("World");  
}
```

OUTPUT

```
Hello  
World
```

However, suppose we would like to print other data types different than string literals, such as integer literals, float literals, or variables. In that case, we need to include a string literal format argument using curly brackets `{}`, a placeholder for our variable(s).

CODE

```
fn main() {  
    // Print an integer using a string literal format argument  
    println!("{}", 7);  
}
```

OUTPUT

```
7
```

We can add multiple placeholders, with the arguments to print separated by a comma `,`:

CODE

```
fn main() {  
    // Multiple placeholders  
    println!("The numbers are: {} and {}", 7, 4);  
}
```

OUTPUT

```
The numbers are: 7 and 4
```

We can also add text with the placeholder to format our output:

CODE

```
fn main() {  
    // Format output with additional text  
    println!("The number is: {}", 7);  
}
```

If we want to print variables, we can do so by including the same string literal format argument as before:

CODE

```
fn main() {  
    let intvar = 7421;  
    println!("{}", intvar);  
}
```

OUTPUT

```
7421
```

§

Data types

We mentioned that Rust is a statically-typed language. This means that all data types must be explicitly known to the interpreter before compiling. We have two options when dealing with data types:

- To explicitly define the data type.
- To let the Rust compiler fill in a default data type based on the variable's assigned value.

When working with data types in Rust, there are some core concepts we need to take into account:

- If we have a variable of some type, `T`, we own the data associated with `T`. If we have a variable of type `&T`, then we don't own the data; we're borrowing it.

There are 17 main data types in Rust, which can in turn, be divided into six categories:

- Primitive types
- Sequence types
- User-defined types
- Function types
- Pointer types
- Trait types

Although we will only cover primitive and sequence types in this segment, the full documentation can be consulted [here](#).

1. Primitive types

Primitives refer to the basic data types built into the language and not defined in terms of other data types. There are four main primitive data types in Rust:

- Boolean - `bool`
- Numeric - integer `i` (*signed or unsigned*) and float `f`
- Textual - `char` and `str`
- Never - `!`

1.1 Boolean

The boolean type can take one of two possible values:

- `true` : Has the bit pattern `0x01`
- `false` : Has the bit pattern `0x00`

We can define a simple boolean variable:

CODE

```
fn main() {
    let boolean_var:bool = true;
    println!("{}", boolean_var);
}
```

OUTPUT

```
true
```

1.1.2 BOOLEAN LOGIC

Boolean values can be operated on using boolean logic or boolean operators. There are five main boolean operators in Rust. Given two variables, `a` and `b`, we can define them as follows:

- Logical not: `!a`
- Logical or: `a | b`
- Logical and: `a & b`
- Logical xor: `a ^ b`
- Logical comparisons:
 - Equal to: `a == b`
 - Greater than: `a > b`
 - Greater than or equal to: `a >= b`
 - Less than: `a < b`
 - Less than or equal to: `a <= b`

1.2 Integer

As mentioned, there are signed and unsigned integer types. The main difference between the two is that:

- A signed integer can hold negative values.
- An unsigned integer can hold a larger positive value and no negative value.

An integer type can also be classified based on its minimum and maximum number of bits (*capacity*), e.g., `i8` (*signed*) will accept a number from -128 to 127, while `i16` (*signed*) will accept a number from -32,768 to 32,767.

This does not just apply to Rust but is common to all programming languages (*it's just that dynamically typed languages handle this automatically for us*) and directly related to the computer hardware and how a system stores information in memory.

Below is the complete list of signed integer data types:

Type	Minimum	Maximum
<code>i8</code>	$-(2^7)$	2^7-1
<code>i16</code>	$-(2^{15})$	$2^{15}-1$
<code>i32</code>	$-(2^{31})$	$2^{31}-1$
<code>i64</code>	$-(2^{63})$	$2^{63}-1$
<code>i128</code>	$-(2^{127})$	$2^{127}-1$

TABLE 4: SIGNED INTEGER TYPES WITH THEIR RESPECTIVE MINIMUM AND MAXIMUM VALUES

As well as their unsigned counterparts:

Type	Minimum	Maximum
<code>u8</code>	0	2^8-1
<code>u16</code>	0	$2^{16}-1$
<code>u32</code>	0	$2^{32}-1$
<code>u64</code>	0	$2^{64}-1$
<code>u128</code>	0	$2^{128}-1$

TABLE 5: UNSIGNED INTEGER TYPES WITH THEIR RESPECTIVE MINIMUM AND MAXIMUM VALUES

It's important to note that the default type for any Rust program will be `i32` (i.e., if we define an integer numeric variable without explicitly stating its data type, the compiler will most probably suggest `i32` as its data type).

We can define an integer variable under the current scope:

CODE

```
fn main() {  
    let mynum_i32:i32 = 2000;  
}
```

We can also directly check the maximum number possible for a given data type using the following syntax:

CODE

```
fn main() {  
    println!("Max size for u8 is: {}", u8::MAX);  
}
```

OUTPUT

```
Max size for u8 is: 255
```

If we try to define a variable with a data type where our value is out of its range, we will get the following error:

CODE

```
fn main() {  
    let mynumber:u8 = 256;  
}
```

OUTPUT

```
literal out of range for `u8`
```

1.3 Float

Similar to integers, floating-point values can be classified by bit size and defined by the [IEEE 754 standard](#). The only difference is that there are only two different types we can declare:

- `f32` : "[*binary32*](#)" type is 32 bits in size and has single precision.
- `f64` : "[*binary64*](#)" type is 64 bits in size and has double precision.

We can define a floating-point variable under the current scope:

CODE

```
fn main() {  
    let mynum_f32:f32 = 3.1416;  
}
```

As with integer types, there is a default floating-point type suggested previously to compilation time; the default type for any Rust program will be `f64` (i.e., if we define a floating-point numeric variable without explicitly stating its data type, the compiler will most probably suggest `f64` as its data type).

We mentioned precision when discussing the floating-point types available in Rust. This attribute is common to all floating-point types denoting the number of digits after the [radix point](#).

Single precision means a limited set of possible numbers after the radix point, while double precision means more possible numbers.

More concretely, single-precision floating-point numbers provide roughly 6 to 7 significant decimal digits, while double-precision numbers provide approximately 14.

1.4 String

Strings are slightly different in terms of how we use them in Rust; strings and slices are only accessible via references (*borrowing*), thus when we declare a string using the `str` data type, for example, we will have to prepend our data type with an ampersand `&`.

A reference consists of a pointer into memory which cannot be changed directly by the owner of the reference since the owner is not the actual owner of the underlying type.

There are two types of strings in Rust:

- `String`: Is stored as a vector of bytes that can be changed but is always guaranteed to be a valid UTF-8 sequence.
- `&str`: Also called string literal, it's a slice that always points to a valid UTF-8 sequence and can be used to view into a `String`.

To define an empty `String`, we can use the following syntax:

CODE

```
fn main() {  
    let mut mystring1 = String::new();  
}
```

We can append to, iterate over, and replace items in a `String`:

CODE

```
fn main() {  
    // Insert a character  
    mystring1.push('X');  
  
    // Insert a string  
    mystring1.push_str("Y Z");  
  
    // Iterate over words by splitting at whitespaces  
    for i in mystring1.split_whitespace() {  
        println!("{}", i);  
    }  
  
    // Replace a string  
    let mystring2: String = mystring1.replace("X", "W X");  
    println!("{}", mystring2);  
}
```

Note that the first entry we pushed was actually a character `char` and not a string `String`. As we'll see shortly, these two data types are different and, thus, are declared differently:

- String: `""`
- Character: `''`

String indexing can be achieved given that we first convert our `String` to a heap-allocated `String`:

CODE

```
fn main() {
    // First convert to heap-allocated string
    let myheapstring: String = mystring2.to_string();
    println!("Heap: {}", myheapstring);

    // Then index
    let mysubstring2 = &myheapstring[1..6];
    println!("Substring: {}", mysubstring2);
}
```

OUTPUT

```
Heap: W XY Z
Substring: XY Z
```

We can also generate a non-empty `String` :

CODE

```
fn main() {
    let mystring3 = String::from("This is is a non-empty string");
}
```

So it seems we included a duplicate `is` string in there, so we might want to eliminate it. We mentioned that a `String` type is stored as a vector of bytes that can be changed. Thus, we can first convert our `String` to a vector and then remove duplicate entries:

CODE

```
fn main() {
    // First convert to vector
    let mut myvec1: Vec<String> = mystring3.split_whitespace().map(str::to_string).collect();
    println!("{:?}", myvec1);

    // Then remove duplicates
    myvec1.dedup();
    println!("{:?}", myvec1);
}
```

OUTPUT

```
["This", "is", "is", "a", "non-empty", "string"]
["This", "is", "a", "non-empty", "string"]
```

Once we have a vector, it's easy to perform various operations & transformations, as we'll see when we get to vectors.

1.5 Character

A character `char` is similar to a string, the main difference being the first is always four bytes, while the latter doesn't have to be composed of just four-byte chunks.

We can declare a char under the current scope by using single quotes `' '`:

CODE

```
fn main() {  
    let mychar:char = 'a';  
}
```

If we try to include more than one character, we will get an error:

CODE

```
fn main() {  
    let mychar:char = 'ab';  
}
```

OUTPUT

```
character literal may only contain one codepoint  
Syntax Error: Literal must be one character long
```

1.6 Never

The never type `!` is a type with no values, representing the result of incomplete computations. As of Rust 1.68.2 (*current version as of April 2023*), the `!` type can only appear in function return types. Expressions of type `!` can be coerced into any other type.

2. Sequence types

There are three main sequence data types in Rust:

- Tuple - `( Type_1 , Type_2, Type_n )`
- Array - `[ Type ; Expression ]`
- Slice - `[ Type ]`

2.1 Tuple

A tuple can have members of different types but cannot change size. Different data types mean we must define each item's types inside the tuple. Also, tuples are immutable by default, but we can define a mutable tuple using the `mut` keyword.

2.1.1 IMMUTABLE TUPLES

A tuple is defined using the following syntax:

CODE